MODULE *DataLakehouseGateway*

EXTENDS *Naturals*, *Sequences*, *FiniteSets*, *TLC*

CONSTANTS
Tenants, *Clients*, *Requests*, *Queries*, *Workers*, *Blobs*, *ProxyNodes*,
MaxSnapshot, *MaxAttempts*, *MaxCrashes*, *MaxTimeouts*, *ScaleOutThreshold*,
RequestedTenantOf,
AuthenticatedTenantOf,
QueryOf, *ClientOf*,
ProtocolOf,
SecurityContextOf,
FlightPopulatesCache,
MutantSkipTenantCheck

ASSUME
$\land$ DOMAIN *RequestedTenantOf* $=$ *Requests*
$\land$ DOMAIN *AuthenticatedTenantOf* $=$ *Requests*
$\land$ DOMAIN *QueryOf* $=$ *Requests*
$\land$ DOMAIN *ClientOf* $=$ *Requests*
$\land$ DOMAIN *ProtocolOf* $=$ *Requests*
$\land$ DOMAIN *SecurityContextOf* $=$ *Requests*
$\land$ *ProxyNodes* $\neq \{\}$
$\land$ *ScaleOutThreshold* $\in$ *Nat*
$\land$ *MaxSnapshot* $\geq 1 \land$ *MaxAttempts* ≥ 1
$\land$ *MaxCrashes* $\in$ *Nat* $\land$ *MaxTimeouts* $\in$ *Nat*
$\land$ *FlightPopulatesCache* $\in$ BOOLEAN
$\land$ *MutantSkipTenantCheck* $\in$ BOOLEAN
$\land$ *Requests* $\cap$ *Workers* $= \{\}$

NONE $\triangleq$ "NONE"

SecCtxs $\triangleq \{$*SecurityContextOf*$[r] : r \in$ *Requests*$\}$
NoEntry $\triangleq$ [*valid* $\mapsto$ FALSE, *blob_id* $\mapsto$ *NONE*]
FreeBlob $\triangleq$ [
allocated $\mapsto$ FALSE,
staged $\mapsto$ FALSE,
req $\mapsto$ *NONE*,
epoch $\mapsto 0$,
tenant $\mapsto$ *NONE*,
query $\mapsto$ *NONE*,
version $\mapsto 0$,
security_context $\mapsto$ *NONE*
]

--algorithm *DataLakehouseGateway*

variables

hash_ring $\in$ [*Tenants* $\times$ *Queries* $\rightarrow$ *ProxyNodes*],

catalog_snapshot $=$ [$t \in$ *Tenants* $\mapsto 1$],

$cache = [p \in ProxyNodes \mapsto$
$[t \in Tenants \mapsto$
$[q \in Queries \mapsto$
$[v \in 1 \,..\, MaxSnapshot \mapsto$
$[s \in SecCtxs \mapsto NoEntry]]]]],$

$task_queue = \langle\rangle,$

$jobs = [r \in Requests \mapsto$
$[state \mapsto$ "IDLE",
$owner \mapsto NONE,$
$lease_epoch \mapsto 0,$
$attempts \mapsto 0,$
$requested_tenant \mapsto RequestedTenantOf[r],$
$tenant \mapsto NONE,$
$query \mapsto QueryOf[r],$
$security_context \mapsto NONE,$
$version \mapsto 0,$
$blob_ptr \mapsto NONE,$
$protocol \mapsto ProtocolOf[r],$
$rls_applied \mapsto$ FALSE,
$executed_predicate_tenant \mapsto NONE]],$

$blob_storage = [b \in Blobs \mapsto FreeBlob],$
$available_blobs = Blobs,$

$worker_status = [w \in Workers \mapsto$ "OFFLINE"$],$

$client_received = [r \in Requests \mapsto$
$[delivered \mapsto$ FALSE,
$released \mapsto$ FALSE,
$blob_id \mapsto NONE,$
$tenant \mapsto NONE,$
$query \mapsto NONE,$
$version \mapsto 0,$
$security_context \mapsto NONE]],$

$commit_count = [r \in Requests \mapsto 0],$
$lease_expired = [r \in Requests \mapsto$ FALSE$],$

$crashes = 0,\ timeouts = 0$ **;**

define
$Pending \triangleq \{$"QUEUED", "CLAIMED", "RUNNING", "STAGED"$\}$
$Active \triangleq \{$"CLAIMED", "RUNNING", "STAGED"$\}$
$Terminal \triangleq \{$"ACCEPTED", "FAILED", "UNAUTHORIZED"$\}$
$AllStates \triangleq \{$"IDLE"$\} \cup Pending \cup Terminal$

$EffTenant(r) \triangleq$
IF $MutantSkipTenantCheck$
THEN $RequestedTenantOf[r]$
ELSE $AuthenticatedTenantOf[r]$

$HoldsLease(r,\ e,\ w,\ st) \triangleq$
$\land\ jobs[r].lease_epoch = e$

$\land\ jobs[r].owner = w$
$\land\ jobs[r].state = st$

$QCount(r) \triangleq Cardinality(\{i \in 1\,..\,Len(task_queue) : task_queue[i] = r\})$

$BlobMatches(b,\ t,\ q,\ v,\ s) \triangleq$
$\land\ blob_storage[b].tenant = t$
$\land\ blob_storage[b].query = q$
$\land\ blob_storage[b].version = v$
$\land\ blob_storage[b].security_context = s$

$BlobCached(b) \triangleq$
$\exists\, p \in ProxyNodes,\ t \in Tenants,\ q \in Queries,$
$v \in 1\,..\,MaxSnapshot,\ s \in SecCtxs :$
$cache[p][t][q][v][s].valid \land cache[p][t][q][v][s].blob_id = b$

$BlobJobHeld(b) \triangleq$
$\exists\, r \in Requests :$
$jobs[r].blob_ptr = b \land \neg client_received[r].released$

$BlobClientHeld(b) \triangleq$
$\exists\, r \in Requests :$
$\land\ client_received[r].delivered$
$\land\ \neg client_received[r].released$
$\land\ client_received[r].blob_id = b$

$StagedOrphan(b) \triangleq$
$jobs[blob_storage[b].req].lease_epoch \neq blob_storage[b].epoch$

$GCable \triangleq \{b \in Blobs :$
$\land\ blob_storage[b].allocated$
$\land\ \text{IF}\ blob_storage[b].staged$
$\ \text{THEN}\ StagedOrphan(b)$
$\ \text{ELSE}\ \neg BlobCached(b) \land \neg BlobJobHeld(b) \land \neg BlobClientHeld(b)\}$

$ExpirableJobs \triangleq \{r \in Requests :$
$\land\ jobs[r].state \in Active$
$\land\ (worker_status[jobs[r].owner] \neq$ "ACTIVE"
$\lor\ lease_expired[r])\}$

$CachedKeys \triangleq \{k \in ProxyNodes \times Tenants \times Queries$
$\times\ (1\,..\,MaxSnapshot) \times SecCtxs :$
$cache[k[1]][k[2]][k[3]][k[4]][k[5]].valid\}$

$TypeOK \triangleq$
$\land\ catalog_snapshot \in [Tenants \to 1\,..\,MaxSnapshot]$
$\land\ worker_status \in [Workers \to \{$"OFFLINE", "ACTIVE", "CRASHED"$\}]$
$\land\ hash_ring \in [Tenants \times Queries \to ProxyNodes]$
$\land\ \forall\, p \in ProxyNodes,\ t \in Tenants,\ q \in Queries,$
$v \in 1\,..\,MaxSnapshot,\ s \in SecCtxs :$
$cache[p][t][q][v][s].valid \in \text{BOOLEAN}$
$\land\ \forall\, r \in Requests :$
$\land\ jobs[r].state \in AllStates$
$\land\ jobs[r].attempts \in 0\,..\,MaxAttempts$

end define ;

fair process *ClientProcess* $\in$ *Requests*
variables *req_version* $= 0$, *proxy* $=$ *NONE* **;**
begin
Start:
if $\neg$*MutantSkipTenantCheck*
$\wedge$ *AuthenticatedTenantOf*[*self*] $\neq$ *RequestedTenantOf*[*self*] **then**
jobs[*self*] := [*jobs*[*self*] EXCEPT !.*state* = "UNAUTHORIZED"] **;**
goto *Finish* **;**
end if ;

BindSnapshot:
req_version := *catalog_snapshot*[*EffTenant*(*self*)] **;**
proxy := *hash_ring*[*EffTenant*(*self*), *QueryOf*[*self*]] **;**

CheckCache:
if *cache*[*proxy*][*EffTenant*(*self*)][*QueryOf*[*self*]]
[*req_version*][*SecurityContextOf*[*self*]].*valid* **then**

client_received[*self*] := [*client_received*[*self*] EXCEPT
!.*delivered* = TRUE,
!.*blob_id* = *cache*[*proxy*][*EffTenant*(*self*)][*QueryOf*[*self*]]
[*req_version*][*SecurityContextOf*[*self*]].*blob_id*,
!.*tenant* = *EffTenant*(*self*),
!.*query* = *QueryOf*[*self*],
!.*version* = *req_version*,
!.*security_context* = *SecurityContextOf*[*self*]] **;**
goto *Finish* **;**
end if ;

InitializeJob:
jobs[*self*] := [*jobs*[*self*] EXCEPT
!.*tenant* = *EffTenant*(*self*),
!.*security_context* = *SecurityContextOf*[*self*],
!.*version* = *req_version*] **;**

Route:
if *ProtocolOf*[*self*] = "FLIGHT" **then**
goto *ExecuteFlight* **;**
else
goto *EnqueueREST* **;**
end if ;

ExecuteFlight:
await *available_blobs* $\neq \{\}$ **;**
with $b \in$ *available_blobs* **do**
available_blobs := *available_blobs* $\setminus \{b\}$ **;**

blob_storage[b] := [*FreeBlob* EXCEPT
!.*allocated* = TRUE,
!.*tenant* = *jobs*[*self*].*tenant*,
!.*query* = *jobs*[*self*].*query*,

```
!.version = jobs[self].version,
!.security_context = jobs[self].security_context] ;

jobs[self] := [jobs[self] EXCEPT
!.state = "ACCEPTED",
!.lease_epoch = 1,
!.attempts = 1,
!.blob_ptr = b,
!.rls_applied = TRUE,
!.executed_predicate_tenant = jobs[self].tenant] ;

commit_count[self] := commit_count[self] + 1 ;

client_received[self] := [client_received[self] EXCEPT
!.delivered = TRUE,
!.blob_id = b,
!.tenant = jobs[self].tenant,
!.query = jobs[self].query,
!.version = jobs[self].version,
!.security_context = jobs[self].security_context] ;

if FlightPopulatesCache
∧ jobs[self].version = catalog_snapshot[jobs[self].tenant] then
cache[proxy][jobs[self].tenant][jobs[self].query]
[jobs[self].version][jobs[self].security_context] :=
[valid ↦ TRUE, blob_id ↦ b] ;
end if ;
end with ;
goto Finish ;

EnqueueREST:
jobs[self] := [jobs[self] EXCEPT
!.state = "QUEUED",
!.attempts = @ + 1] ;
task_queue := Append(task_queue, self) ;

AwaitREST:
await jobs[self].state ∈ Terminal ;
if jobs[self].state = "ACCEPTED" then
client_received[self] := [client_received[self] EXCEPT
!.delivered = TRUE,
!.blob_id = jobs[self].blob_ptr,
!.tenant = jobs[self].tenant,
!.query = jobs[self].query,
!.version = jobs[self].version,
!.security_context = jobs[self].security_context] ;
end if ;

Finish:
skip ;
end process ;

fair process ClientReleaser ∈ Requests
begin
Release:
await client_received[self].delivered ∧ ¬client_received[self].released ;
client_received[self] := [client_received[self] EXCEPT
```

```
!.released = TRUE] ;
end process ;



fair process ComputeWorker ∈ Workers
variables curr_req = NONE, curr_epoch = 0, curr_blob = NONE, curr_proxy = NONE ;
begin
ClaimTask:
while TRUE do
await Len(task_queue) > 0 ∧ worker_status[self] = "ACTIVE" ;
with r = Head(task_queue) do
task_queue := Tail(task_queue) ;
curr_req := r ;
curr_blob := NONE ;
if jobs[r].state = "QUEUED" then
curr_epoch := jobs[r].lease_epoch + 1 ;
jobs[r] := [jobs[r] EXCEPT
!.owner = self,
!.state = "CLAIMED",
!.lease_epoch = @ + 1] ;
else
curr_epoch := 0 ;
end if ;
end with ;

StartExecution:
if curr_epoch > 0
∧ worker_status[self] = "ACTIVE"
∧ HoldsLease(curr_req, curr_epoch, self, "CLAIMED") then
jobs[curr_req] := [jobs[curr_req] EXCEPT
!.state = "RUNNING",
!.rls_applied = TRUE,
!.executed_predicate_tenant = jobs[curr_req].tenant] ;
else
goto ClaimTask ;
end if ;

AllocateBlob:
await worker_status[self] ≠ "ACTIVE"
∨ ¬HoldsLease(curr_req, curr_epoch, self, "RUNNING")
∨ available_blobs ≠ {} ;

if worker_status[self] = "ACTIVE"
∧ HoldsLease(curr_req, curr_epoch, self, "RUNNING")
∧ available_blobs ≠ {} then
with b ∈ available_blobs do
available_blobs := available_blobs \ {b} ;
curr_blob := b ;

blob_storage[b] := [FreeBlob EXCEPT
!.allocated = TRUE,
!.staged = TRUE,
!.req = curr_req,
!.epoch = curr_epoch,
```

```
!.tenant = jobs[curr_req].tenant,
!.query = jobs[curr_req].query,
!.version = jobs[curr_req].version,
!.security_context = jobs[curr_req].security_context] ;
end with ;
else
goto ClaimTask ;
end if ;

PublishPointer:
if worker_status[self] = "ACTIVE"
∧ HoldsLease(curr_req, curr_epoch, self, "RUNNING") then
jobs[curr_req] := [jobs[curr_req] EXCEPT
!.blob_ptr = curr_blob,
!.state = "STAGED"] ;
else
goto ClaimTask ;
end if ;

CommitJob:
if worker_status[self] = "ACTIVE"
∧ HoldsLease(curr_req, curr_epoch, self, "STAGED") then
jobs[curr_req] := [jobs[curr_req] EXCEPT
!.state = "ACCEPTED",
!.owner = NONE] ;

blob_storage[curr_blob] := [blob_storage[curr_blob] EXCEPT
!.staged = FALSE] ;

commit_count[curr_req] := commit_count[curr_req] + 1 ;
else
goto ClaimTask ;
end if ;

PopulateCache:
if worker_status[self] = "ACTIVE"
∧ jobs[curr_req].state = "ACCEPTED"
∧ jobs[curr_req].blob_ptr = curr_blob
∧ jobs[curr_req].version = catalog_snapshot[jobs[curr_req].tenant] then

curr_proxy := hash_ring[jobs[curr_req].tenant, jobs[curr_req].query] ;

cache[curr_proxy][jobs[curr_req].tenant][jobs[curr_req].query]
[jobs[curr_req].version][jobs[curr_req].security_context] :=
[valid ↦ TRUE, blob_id ↦ curr_blob] ;
end if ;
end while ;
end process ;




fair process KedaAutoscaler = "KedaAutoscaler"
begin
ScaleLoop:
while TRUE do
```

either

await $Len(task_queue) \geq ScaleOutThreshold$
$\land \exists\, w \in Workers : worker_status[w] =$ "OFFLINE" **;**

with $w \in \{x \in Workers : worker_status[x] =$ "OFFLINE"$\}$ **do**
$worker_status[w] :=$ "ACTIVE" **;**
end with ;
or

await $Len(task_queue) = 0$
$\land \exists\, w \in Workers :$
$worker_status[w] =$ "ACTIVE"
$\land \forall\, r \in Requests :$
$\neg(jobs[r].owner = w \land jobs[r].state \in Active)$ **;**

with $w \in \{x \in Workers :$
$worker_status[x] =$ "ACTIVE"
$\land \forall\, r \in Requests :$
$\neg(jobs[r].owner = x \land jobs[r].state \in Active)\}$ **do**
$worker_status[w] :=$ "OFFLINE" **;**
end with ;
end either ;
end while ;
end process ;

process $TimeoutInjector =$ "TimeoutInjector"
begin
InjectLoop:
while $timeouts < MaxTimeouts$ **do**
with $r \in \{x \in Requests :$
$jobs[x].state \in Active$
$\land\, worker_status[jobs[x].owner] =$ "ACTIVE"
$\land \neg lease_expired[x]\}$ **do**
$lease_expired[r] :=$ TRUE **;**
$timeouts := timeouts + 1$ **;**
end with ;
end while ;
end process ;

fair process $LeaseManager =$ "LeaseManager"
begin
LeaseLoop:
while TRUE **do**
with $r \in ExpirableJobs$ **do**
if $jobs[r].attempts < MaxAttempts$ **then**
$jobs[r] := [jobs[r]$ EXCEPT
$!.owner = NONE,$
$!.state =$ "QUEUED",

!.*attempts* = @ + 1,
!.*lease_epoch* = @ + 1,
!.*blob_ptr* = *NONE*,
!.*rls_applied* = FALSE,
!.*executed_predicate_tenant* = *NONE*] ;
task_queue := *Append*(*task_queue*, *r*) ;
else
jobs[*r*] := [*jobs*[*r*] EXCEPT
!.*owner* = *NONE*,
!.*state* = "FAILED",
!.*lease_epoch* = @ + 1,
!.*blob_ptr* = *NONE*,
!.*rls_applied* = FALSE,
!.*executed_predicate_tenant* = *NONE*] ;
end if ;
lease_expired[*r*] := FALSE ;
end with ;
end while ;
end process ;

process *Crasher* = "Crasher"
begin
CrashLoop:
while *crashes* < *MaxCrashes* **do**
await ∃ *w* ∈ *Workers* : *worker_status*[*w*] = "ACTIVE" ;
with *w* ∈ {*x* ∈ *Workers* : *worker_status*[*x*] = "ACTIVE"} **do**
worker_status[*w*] := "CRASHED" ;
crashes := *crashes* + 1 ;
end with ;
end while ;
end process ;

fair process *Recoverer* = "Recoverer"
begin
RecoverLoop:
while TRUE **do**
await ∃ *w* ∈ *Workers* : *worker_status*[*w*] = "CRASHED" ;
with *w* ∈ {*x* ∈ *Workers* : *worker_status*[*x*] = "CRASHED"} **do**

worker_status[*w*] := "OFFLINE" ;
end with ;
end while ;
end process ;

fair process *GarbageCollector* = "GarbageCollector"
begin
GCLoop:
while TRUE **do**
with *b* ∈ *GCable* **do**

```
blob_storage[b] := FreeBlob ;
available_blobs := available_blobs ∪ {b} ;
end with ;
end while ;
end process ;

fair process CacheEvictor = "CacheEvictor"
begin
EvictLoop:
while TRUE do
with e ∈ CachedKeys do
cache[e[1]][e[2]][e[3]][e[4]][e[5]] := NoEntry ;
end with ;
end while ;
end process ;

fair process LakehouseETL = "LakehouseETL"
begin
ETLCommit:
while TRUE do
with t ∈ {x ∈ Tenants : catalog_snapshot[x] < MaxSnapshot} do

cache := [p ∈ ProxyNodes ↦
[cache[p] EXCEPT ![t] =
[q ∈ Queries ↦
[v ∈ 1 .. MaxSnapshot ↦
[s ∈ SecCtxs ↦
IF v < catalog_snapshot[t] + 1
 THEN NoEntry
 ELSE cache[p][t][q][v][s]]]]] ;

catalog_snapshot[t] := catalog_snapshot[t] + 1 ;
end with ;
end while ;
end process ;

end algorithm ;

VARIABLES pc, hash_ring, catalog_snapshot, cache, task_queue, jobs,
blob_storage, available_blobs, worker_status, client_received,
commit_count, lease_expired, crashes, timeouts


Pending ≜ {"QUEUED", "CLAIMED", "RUNNING", "STAGED"}
Active ≜ {"CLAIMED", "RUNNING", "STAGED"}
Terminal ≜ {"ACCEPTED", "FAILED", "UNAUTHORIZED"}
AllStates ≜ {"IDLE"} ∪ Pending ∪ Terminal

EffTenant(r) ≜
IF MutantSkipTenantCheck
 THEN RequestedTenantOf[r]
 ELSE AuthenticatedTenantOf[r]

HoldsLease(r, e, w, st) ≜
∧ jobs[r].lease_epoch = e
∧ jobs[r].owner = w
∧ jobs[r].state = st
```

$QCount(r) \triangleq Cardinality(\{i \in 1 \,..\, Len(task_queue) : task_queue[i] = r\})$

$BlobMatches(b,\ t,\ q,\ v,\ s) \triangleq$
$\land\ blob_storage[b].tenant = t$
$\land\ blob_storage[b].query = q$
$\land\ blob_storage[b].version = v$
$\land\ blob_storage[b].security_context = s$

$BlobCached(b) \triangleq$
$\exists\, p \in ProxyNodes,\ t \in Tenants,\ q \in Queries,$
$v \in 1 \,..\, MaxSnapshot,\ s \in SecCtxs :$
$cache[p][t][q][v][s].valid \land cache[p][t][q][v][s].blob_id = b$

$BlobJobHeld(b) \triangleq$
$\exists\, r \in Requests :$
$jobs[r].blob_ptr = b \land \neg client_received[r].released$

$BlobClientHeld(b) \triangleq$
$\exists\, r \in Requests :$
$\land\ client_received[r].delivered$
$\land \neg client_received[r].released$
$\land\ client_received[r].blob_id = b$

$StagedOrphan(b) \triangleq$
$jobs[blob_storage[b].req].lease_epoch \neq blob_storage[b].epoch$

$GCable \triangleq \{b \in Blobs :$
$\land\ blob_storage[b].allocated$
$\land$ IF $blob_storage[b].staged$
THEN $StagedOrphan(b)$
ELSE $\neg BlobCached(b) \land \neg BlobJobHeld(b) \land \neg BlobClientHeld(b)\}$

$ExpirableJobs \triangleq \{r \in Requests :$
$\land\ jobs[r].state \in Active$
$\land\ (worker_status[jobs[r].owner] \neq$ "ACTIVE"
$\lor\ lease_expired[r])\}$

$CachedKeys \triangleq \{k \in ProxyNodes \times Tenants \times Queries$
$\times\, (1 \,..\, MaxSnapshot) \times SecCtxs :$
$cache[k[1]][k[2]][k[3]][k[4]][k[5]].valid\}$

$TypeOK \triangleq$
$\land\ catalog_snapshot \in [Tenants \to 1 \,..\, MaxSnapshot]$
$\land\ worker_status \in [Workers \to \{$"OFFLINE", "ACTIVE", "CRASHED"$\}]$
$\land\ hash_ring \in [Tenants \times Queries \to ProxyNodes]$
$\land \forall\, p \in ProxyNodes,\ t \in Tenants,\ q \in Queries,$
$v \in 1 \,..\, MaxSnapshot,\ s \in SecCtxs :$
$cache[p][t][q][v][s].valid \in$ BOOLEAN
$\land \forall\, r \in Requests :$
$\land\ jobs[r].state \in AllStates$
$\land\ jobs[r].attempts \in 0 \,..\, MaxAttempts$

VARIABLES $req_version$, $proxy$, $curr_req$, $curr_epoch$, $curr_blob$, $curr_proxy$

$vars \triangleq \langle pc,\ hash_ring,\ catalog_snapshot,\ cache,\ task_queue,\ jobs,$
$blob_storage,\ available_blobs,\ worker_status,\ client_received,$
$commit_count,\ lease_expired,\ crashes,\ timeouts,\ req_version,\ proxy,$
$curr_req,\ curr_epoch,\ curr_blob,\ curr_proxy\rangle$

$ProcSet \triangleq (Requests) \cup (Requests) \cup (Workers) \cup \{\text{“KedaAutoscaler”}\} \cup \{\text{“TimeoutInjector”}\} \cup \{\text{“LeaseManager”}\} \cup \{\text{“Crasher”}\} \cup$

$Init \triangleq$ Global variables
$\land\ hash_ring \in [Tenants \times Queries \to ProxyNodes]$
$\land\ catalog_snapshot = [t \in Tenants \mapsto 1]$
$\land\ cache = [p \in ProxyNodes \mapsto$
$[t \in Tenants \mapsto$
$[q \in Queries \mapsto$
$[v \in 1\ldots MaxSnapshot \mapsto$
$[s \in SecCtxs \mapsto NoEntry]]]]]$
$\land\ task_queue = \langle\rangle$
$\land\ jobs = [r \in Requests \mapsto$
$[state \mapsto \text{“IDLE”},$
$owner \mapsto NONE,$
$lease_epoch \mapsto 0,$
$attempts \mapsto 0,$
$requested_tenant \mapsto RequestedTenantOf[r],$
$tenant \mapsto NONE,$
$query \mapsto QueryOf[r],$
$security_context \mapsto NONE,$
$version \mapsto 0,$
$blob_ptr \mapsto NONE,$
$protocol \mapsto ProtocolOf[r],$
$rls_applied \mapsto \text{FALSE},$
$executed_predicate_tenant \mapsto NONE]]$
$\land\ blob_storage = [b \in Blobs \mapsto FreeBlob]$
$\land\ available_blobs = Blobs$
$\land\ worker_status = [w \in Workers \mapsto \text{“OFFLINE”}]$
$\land\ client_received = [r \in Requests \mapsto$
$[delivered \mapsto \text{FALSE},$
$released \mapsto \text{FALSE},$
$blob_id \mapsto NONE,$
$tenant \mapsto NONE,$
$query \mapsto NONE,$
$version \mapsto 0,$
$security_context \mapsto NONE]]$
$\land\ commit_count = [r \in Requests \mapsto 0]$
$\land\ lease_expired = [r \in Requests \mapsto \text{FALSE}]$
$\land\ crashes = 0$
$\land\ timeouts = 0$

$\land\ req_version = [self \in Requests \mapsto 0]$
$\land\ proxy = [self \in Requests \mapsto NONE]$

$\land\ curr_req = [self \in Workers \mapsto NONE]$
$\land\ curr_epoch = [self \in Workers \mapsto 0]$
$\land\ curr_blob = [self \in Workers \mapsto NONE]$
$\land\ curr_proxy = [self \in Workers \mapsto NONE]$
$\land\ pc = [self \in ProcSet \mapsto \text{CASE } self \in Requests \to \text{“Start”}$
$\square self \in Requests \to \text{“Release”}$
$\square self \in Workers \to \text{“ClaimTask”}$

$\Box self = \text{“KedaAutoscaler”} \rightarrow \text{“ScaleLoop”}$
$\Box self = \text{“TimeoutInjector”} \rightarrow \text{“InjectLoop”}$
$\Box self = \text{“LeaseManager”} \rightarrow \text{“LeaseLoop”}$
$\Box self = \text{“Crasher”} \rightarrow \text{“CrashLoop”}$
$\Box self = \text{“Recoverer”} \rightarrow \text{“RecoverLoop”}$
$\Box self = \text{“GarbageCollector”} \rightarrow \text{“GCLoop”}$
$\Box self = \text{“CacheEvictor”} \rightarrow \text{“EvictLoop”}$
$\Box self = \text{“LakehouseETL”} \rightarrow \text{“ETLCommit”}]$

$Start(self) \triangleq \ \land pc[self] = \text{“Start”}$
$\land$ IF $\neg MutantSkipTenantCheck$
$\land AuthenticatedTenantOf[self] \neq RequestedTenantOf[self]$
THEN $\land jobs' = [jobs$ EXCEPT $![self] = [jobs[self]$ EXCEPT $!.state = \text{“UNAUTHORIZED”}]]$
$\land pc' = [pc$ EXCEPT $![self] = \text{“Finish”}]$
ELSE $\land pc' = [pc$ EXCEPT $![self] = \text{“BindSnapshot”}]$
$\land jobs' = jobs$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache, task_queue,$
$blob_storage, available_blobs, worker_status,$
$client_received, commit_count, lease_expired,$
$crashes, timeouts, req_version, proxy, curr_req,$
$curr_epoch, curr_blob, curr_proxy\rangle$

$BindSnapshot(self) \triangleq \ \land pc[self] = \text{“BindSnapshot”}$
$\land req_version' = [req_version$ EXCEPT $![self] = catalog_snapshot[EffTenant(self)]]$
$\land proxy' = [proxy$ EXCEPT $![self] = hash_ring[EffTenant(self), QueryOf[self]]]$
$\land pc' = [pc$ EXCEPT $![self] = \text{“CheckCache”}]$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$task_queue, jobs, blob_storage,$
$available_blobs, worker_status,$
$client_received, commit_count,$
$lease_expired, crashes, timeouts,$
$curr_req, curr_epoch, curr_blob,$
$curr_proxy\rangle$

$CheckCache(self) \triangleq \ \land pc[self] = \text{“CheckCache”}$
$\land$ IF $cache[proxy[self]][EffTenant(self)][QueryOf[self]]$
$[req_version[self]][SecurityContextOf[self]].valid$
THEN $\land client_received' = [client_received$ EXCEPT $![self] = [client_received[self]$ EXCEPT
$!.delivered =$ TRUE,
$!.blob_id = cache[proxy[self]][EffTenant(self)][QueryOf[self]]$
$[req_version[self]][SecurityContextOf[self]].blob_id,$
$!.tenant = EffTenant(self),$
$!.query = QueryOf[self],$
$!.version = req_version[self],$
$!.security_context = SecurityContextOf[self]]]$
$\land pc' = [pc$ EXCEPT $![self] = \text{“Finish”}]$
ELSE $\land pc' = [pc$ EXCEPT $![self] = \text{“InitializeJob”}]$
$\land$ UNCHANGED $client_received$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$task_queue, jobs, blob_storage,$
$available_blobs, worker_status,$
$commit_count, lease_expired, crashes,$
$timeouts, req_version, proxy, curr_req,$
$curr_epoch, curr_blob, curr_proxy\rangle$

$InitializeJob(self) \triangleq \ \land pc[self] = \text{“InitializeJob”}$
$\land jobs' = [jobs$ EXCEPT $![self] = [jobs[self]$ EXCEPT

$!.tenant = EffTenant(self)$,
$!.security_context = SecurityContextOf[self]$,
$!.version = req_version[self]]]$
$\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“Route”}]$
$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ cache,$
$task_queue,\ blob_storage,$
$available_blobs,\ worker_status,$
$client_received,\ commit_count,$
$lease_expired,\ crashes,\ timeouts,$
$req_version,\ proxy,\ curr_req,$
$curr_epoch,\ curr_blob,\ curr_proxy\rangle$

$Route(self) \triangleq\ \land pc[self] = \text{“Route”}$
$\land$ IF $ProtocolOf[self] = \text{“FLIGHT”}$
THEN $\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“ExecuteFlight”}]$
ELSE $\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“EnqueueREST”}]$
$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ cache,\ task_queue,$
$jobs,\ blob_storage,\ available_blobs,$
$worker_status,\ client_received,\ commit_count,$
$lease_expired,\ crashes,\ timeouts,\ req_version,$
$proxy,\ curr_req,\ curr_epoch,\ curr_blob,$
$curr_proxy\rangle$

$ExecuteFlight(self) \triangleq\ \land pc[self] = \text{“ExecuteFlight”}$
$\land\ available_blobs \neq \{\}$
$\land \exists\ b \in available_blobs :$
$\land\ available_blobs' = available_blobs \setminus \{b\}$
$\land\ blob_storage' = [blob_storage \text{ EXCEPT } ![b] = [FreeBlob \text{ EXCEPT}$
$!.allocated = \text{TRUE},$
$!.tenant = jobs[self].tenant,$
$!.query = jobs[self].query,$
$!.version = jobs[self].version,$
$!.security_context = jobs[self].security_context]]$
$\land\ jobs' = [jobs \text{ EXCEPT } ![self] = [jobs[self] \text{ EXCEPT}$
$!.state = \text{“ACCEPTED”},$
$!.lease_epoch = 1,$
$!.attempts = 1,$
$!.blob_ptr = b,$
$!.rls_applied = \text{TRUE},$
$!.executed_predicate_tenant = jobs[self].tenant]]$
$\land\ commit_count' = [commit_count \text{ EXCEPT } ![self] = commit_count[self] + 1]$
$\land\ client_received' = [client_received \text{ EXCEPT } ![self] = [client_received[self] \text{ EXCEPT}$
$!.delivered = \text{TRUE},$
$!.blob_id = b,$
$!.tenant = jobs'[self].tenant,$
$!.query = jobs'[self].query,$
$!.version = jobs'[self].version,$
$!.security_context = jobs'[self].security_context]]$
$\land$ IF $FlightPopulatesCache$
$\land\ jobs'[self].version = catalog_snapshot[jobs'[self].tenant]$
THEN $\land\ cache' = [cache \text{ EXCEPT } ![proxy[self]][jobs'[self].tenant][jobs'[self].query]$
$[jobs'[self].version][jobs'[self].security_context] = [valid \mapsto \text{TRUE},\ blob_id \mapsto b]]$
ELSE $\land$ TRUE
$\land\ cache' = cache$
$\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“Finish”}]$
$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ task_queue,$
$worker_status,\ lease_expired,\ crashes,$

$timeouts$, $req_version$, $proxy$, $curr_req$,
$curr_epoch$, $curr_blob$, $curr_proxy\rangle$

$EnqueueREST(self) \triangleq \ \wedge pc[self] = \text{“EnqueueREST”}$
$\wedge jobs' = [jobs \text{ EXCEPT } ![self] = [jobs[self] \text{ EXCEPT}$
$!.state = \text{“QUEUED”},$
$!.attempts = @ + 1]]$
$\wedge task_queue' = Append(task_queue, self)$
$\wedge pc' = [pc \text{ EXCEPT } ![self] = \text{“AwaitREST”}]$
$\wedge \text{UNCHANGED } \langle hash_ring, catalog_snapshot, cache,$
$blob_storage, available_blobs,$
$worker_status, client_received,$
$commit_count, lease_expired, crashes,$
$timeouts, req_version, proxy, curr_req,$
$curr_epoch, curr_blob, curr_proxy\rangle$

$AwaitREST(self) \triangleq \ \wedge pc[self] = \text{“AwaitREST”}$
$\wedge jobs[self].state \in Terminal$
$\wedge \text{IF } jobs[self].state = \text{“ACCEPTED”}$
$\text{THEN } \wedge client_received' = [client_received \text{ EXCEPT } ![self] = [client_received[self] \text{ EXCEPT}$
$!.delivered = \text{TRUE},$
$!.blob_id = jobs[self].blob_ptr,$
$!.tenant = jobs[self].tenant,$
$!.query = jobs[self].query,$
$!.version = jobs[self].version,$
$!.security_context = jobs[self].security_context]]$
$\text{ELSE } \wedge \text{TRUE}$
$\wedge \text{UNCHANGED } client_received$
$\wedge pc' = [pc \text{ EXCEPT } ![self] = \text{“Finish”}]$
$\wedge \text{UNCHANGED } \langle hash_ring, catalog_snapshot, cache,$
$task_queue, jobs, blob_storage,$
$available_blobs, worker_status,$
$commit_count, lease_expired, crashes,$
$timeouts, req_version, proxy, curr_req,$
$curr_epoch, curr_blob, curr_proxy\rangle$

$Finish(self) \triangleq \ \wedge pc[self] = \text{“Finish”}$
$\wedge \text{TRUE}$
$\wedge pc' = [pc \text{ EXCEPT } ![self] = \text{“Done”}]$
$\wedge \text{UNCHANGED } \langle hash_ring, catalog_snapshot, cache, task_queue,$
$jobs, blob_storage, available_blobs,$
$worker_status, client_received, commit_count,$
$lease_expired, crashes, timeouts, req_version,$
$proxy, curr_req, curr_epoch, curr_blob,$
$curr_proxy\rangle$

$ClientProcess(self) \triangleq \ Start(self) \vee BindSnapshot(self)$
$\vee CheckCache(self) \vee InitializeJob(self)$
$\vee Route(self) \vee ExecuteFlight(self)$
$\vee EnqueueREST(self) \vee AwaitREST(self)$
$\vee Finish(self)$

$Release(self) \triangleq \ \wedge pc[self] = \text{“Release”}$
$\wedge client_received[self].delivered \wedge \neg client_received[self].released$
$\wedge client_received' = [client_received \text{ EXCEPT } ![self] = [client_received[self] \text{ EXCEPT}$
$!.released = \text{TRUE}]]$
$\wedge pc' = [pc \text{ EXCEPT } ![self] = \text{“Done”}]$

$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ cache,$
$task_queue,\ jobs,\ blob_storage,$
$available_blobs,\ worker_status,\ commit_count,$
$lease_expired,\ crashes,\ timeouts,\ req_version,$
$proxy,\ curr_req,\ curr_epoch,\ curr_blob,$
$curr_proxy\rangle$

$ClientRereleaser(self) \triangleq Release(self)$

$ClaimTask(self) \triangleq \land pc[self] =$ "ClaimTask"
$\land Len(task_queue) > 0 \land worker_status[self] =$ "ACTIVE"
$\land$ LET $r \triangleq Head(task_queue)$IN
$\land task_queue' = Tail(task_queue)$
$\land curr_req' = [curr_req$ EXCEPT $![self] = r]$
$\land curr_blob' = [curr_blob$ EXCEPT $![self] = NONE]$
$\land$ IF $jobs[r].state =$ "QUEUED"
THEN $\land curr_epoch' = [curr_epoch$ EXCEPT $![self] = jobs[r].lease_epoch + 1]$
$\land jobs' = [jobs$ EXCEPT $![r] = [jobs[r]$ EXCEPT
$!.owner = self,$
$!.state =$ "CLAIMED",
$!.lease_epoch = @ + 1]]$
ELSE $\land curr_epoch' = [curr_epoch$ EXCEPT $![self] = 0]$
$\land jobs' = jobs$
$\land pc' = [pc$ EXCEPT $![self] =$ "StartExecution"$]$
$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ cache,$
$blob_storage,\ available_blobs,$
$worker_status,\ client_received,$
$commit_count,\ lease_expired,\ crashes,$
$timeouts,\ req_version,\ proxy,\ curr_proxy\rangle$

$StartExecution(self) \triangleq \land pc[self] =$ "StartExecution"
$\land$ IF $curr_epoch[self] > 0$
$\land worker_status[self] =$ "ACTIVE"
$\land HoldsLease(curr_req[self],\ curr_epoch[self],\ self,$ "CLAIMED"$)$
THEN $\land jobs' = [jobs$ EXCEPT $![curr_req[self]] = [jobs[curr_req[self]]$ EXCEPT
$!.state =$ "RUNNING",
$!.rls_applied =$ TRUE,
$!.executed_predicate_tenant = jobs[curr_req[self]].tenant]]$
$\land pc' = [pc$ EXCEPT $![self] =$ "AllocateBlob"$]$
ELSE $\land pc' = [pc$ EXCEPT $![self] =$ "ClaimTask"$]$
$\land jobs' = jobs$
$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ cache,$
$task_queue,\ blob_storage,$
$available_blobs,\ worker_status,$
$client_received,\ commit_count,$
$lease_expired,\ crashes,\ timeouts,$
$req_version,\ proxy,\ curr_req,$
$curr_epoch,\ curr_blob,\ curr_proxy\rangle$

$AllocateBlob(self) \triangleq \land pc[self] =$ "AllocateBlob"
$\land worker_status[self] \neq$ "ACTIVE"
$\lor \neg HoldsLease(curr_req[self],\ curr_epoch[self],\ self,$ "RUNNING"$)$
$\lor available_blobs \neq \{\}$
$\land$ IF $worker_status[self] =$ "ACTIVE"
$\land HoldsLease(curr_req[self],\ curr_epoch[self],\ self,$ "RUNNING"$)$
$\land available_blobs \neq \{\}$
THEN $\land \exists\, b \in available_blobs :$

$\land\ available_blobs' = available_blobs \setminus \{b\}$
$\land\ curr_blob' = [curr_blob \text{ EXCEPT } ![self] = b]$
$\land\ blob_storage' = [blob_storage \text{ EXCEPT } ![b] = [FreeBlob \text{ EXCEPT }$
$!.allocated = \text{TRUE},$
$!.staged = \text{TRUE},$
$!.req = curr_req[self],$
$!.epoch = curr_epoch[self],$
$!.tenant = jobs[curr_req[self]].tenant,$
$!.query = jobs[curr_req[self]].query,$
$!.version = jobs[curr_req[self]].version,$
$!.security_context = jobs[curr_req[self]].security_context]]$
$\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“PublishPointer”}]$
ELSE $\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“ClaimTask”}]$
$\land$ UNCHANGED $\langle blob_storage, available_blobs,$
$curr_blob\rangle$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$task_queue, jobs, worker_status,$
$client_received, commit_count,$
$lease_expired, crashes, timeouts,$
$req_version, proxy, curr_req, curr_epoch,$
$curr_proxy\rangle$

$PublishPointer(self) \triangleq \land\ pc[self] = \text{“PublishPointer”}$
$\land$ IF $worker_status[self] = \text{“ACTIVE”}$
$\land\ HoldsLease(curr_req[self], curr_epoch[self], self, \text{“RUNNING”})$
THEN $\land\ jobs' = [jobs \text{ EXCEPT } ![curr_req[self]] = [jobs[curr_req[self]] \text{ EXCEPT }$
$!.blob_ptr = curr_blob[self],$
$!.state = \text{“STAGED”}]]$
$\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“CommitJob”}]$
ELSE $\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“ClaimTask”}]$
$\land\ jobs' = jobs$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$task_queue, blob_storage,$
$available_blobs, worker_status,$
$client_received, commit_count,$
$lease_expired, crashes, timeouts,$
$req_version, proxy, curr_req,$
$curr_epoch, curr_blob, curr_proxy\rangle$

$CommitJob(self) \triangleq \land\ pc[self] = \text{“CommitJob”}$
$\land$ IF $worker_status[self] = \text{“ACTIVE”}$
$\land\ HoldsLease(curr_req[self], curr_epoch[self], self, \text{“STAGED”})$
THEN $\land\ jobs' = [jobs \text{ EXCEPT } ![curr_req[self]] = [jobs[curr_req[self]] \text{ EXCEPT }$
$!.state = \text{“ACCEPTED”},$
$!.owner = NONE]]$
$\land\ blob_storage' = [blob_storage \text{ EXCEPT } ![curr_blob[self]] = [blob_storage[curr_blob[self]] \text{ EXCEPT }$
$!.staged = \text{FALSE}]]$
$\land\ commit_count' = [commit_count \text{ EXCEPT } ![curr_req[self]] = commit_count[curr_req[self]] + 1]$
$\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“PopulateCache”}]$
ELSE $\land\ pc' = [pc \text{ EXCEPT } ![self] = \text{“ClaimTask”}]$
$\land$ UNCHANGED $\langle jobs, blob_storage, commit_count\rangle$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache,$
$task_queue, available_blobs, worker_status,$
$client_received, lease_expired, crashes,$
$timeouts, req_version, proxy, curr_req,$
$curr_epoch, curr_blob, curr_proxy\rangle$

$PopulateCache(self) \triangleq \land pc[self] = \text{“PopulateCache”}$
$\land$ IF $worker_status[self] = \text{“ACTIVE”}$
$\land jobs[curr_req[self]].state = \text{“ACCEPTED”}$
$\land jobs[curr_req[self]].blob_ptr = curr_blob[self]$
$\land jobs[curr_req[self]].version = catalog_snapshot[jobs[curr_req[self]].tenant]$
THEN $\land curr_proxy' = [curr_proxy$ EXCEPT $![self] = hash_ring[jobs[curr_req[self]].tenant, jobs[curr_req[self]].query]$
$\land cache' = [cache$ EXCEPT $![curr_proxy'[self]][jobs[curr_req[self]].tenant][jobs[curr_req[self]].query]$
$[jobs[curr_req[self]].version][jobs[curr_req[self]].security_context] = [valid \mapsto$ TRUE, $blob_id \mapsto curr_blob[self]]]$
ELSE $\land$ TRUE
$\land$ UNCHANGED $\langle cache, curr_proxy\rangle$
$\land pc' = [pc$ EXCEPT $![self] = \text{“ClaimTask”}]$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, task_queue,$
$jobs, blob_storage, available_blobs,$
$worker_status, client_received,$
$commit_count, lease_expired, crashes,$
$timeouts, req_version, proxy, curr_req,$
$curr_epoch, curr_blob\rangle$

$ComputeWorker(self) \triangleq ClaimTask(self) \lor StartExecution(self)$
$\lor AllocateBlob(self) \lor PublishPointer(self)$
$\lor CommitJob(self) \lor PopulateCache(self)$

$ScaleLoop \triangleq \land pc[\text{“KedaAutoscaler”}] = \text{“ScaleLoop”}$
$\land \lor \land Len(task_queue) \geq ScaleOutThreshold$
$\land \exists\, w \in Workers : worker_status[w] = \text{“OFFLINE”}$
$\land \exists\, w \in \{x \in Workers : worker_status[x] = \text{“OFFLINE”}\} :$
$worker_status' = [worker_status$ EXCEPT $![w] = \text{“ACTIVE”}]$
$\lor \land Len(task_queue) = 0$
$\land \exists\, w \in Workers :$
$worker_status[w] = \text{“ACTIVE”}$
$\land \forall\, r \in Requests :$
$\neg(jobs[r].owner = w \land jobs[r].state \in Active)$
$\land \exists\, w \in \{x \in Workers :$
$worker_status[x] = \text{“ACTIVE”}$
$\land \forall\, r \in Requests :$
$\neg(jobs[r].owner = x \land jobs[r].state \in Active)\} :$
$worker_status' = [worker_status$ EXCEPT $![w] = \text{“OFFLINE”}]$
$\land pc' = [pc$ EXCEPT $![\text{“KedaAutoscaler”}] = \text{“ScaleLoop”}]$
$\land$ UNCHANGED $\langle hash_ring, catalog_snapshot, cache, task_queue,$
$jobs, blob_storage, available_blobs,$
$client_received, commit_count, lease_expired,$
$crashes, timeouts, req_version, proxy, curr_req,$
$curr_epoch, curr_blob, curr_proxy\rangle$

$KedaAutoscaler \triangleq ScaleLoop$

$InjectLoop \triangleq \land pc[\text{“TimeoutInjector”}] = \text{“InjectLoop”}$
$\land$ IF $timeouts < MaxTimeouts$
THEN $\land \exists\, r \in \{x \in Requests :$
$jobs[x].state \in Active$
$\land worker_status[jobs[x].owner] = \text{“ACTIVE”}$
$\land \neg lease_expired[x]\} :$
$\land lease_expired' = [lease_expired$ EXCEPT $![r] =$ TRUE$]$
$\land timeouts' = timeouts + 1$
$\land pc' = [pc$ EXCEPT $![\text{“TimeoutInjector”}] = \text{“InjectLoop”}]$
ELSE $\land pc' = [pc$ EXCEPT $![\text{“TimeoutInjector”}] = \text{“Done”}]$
$\land$ UNCHANGED $\langle lease_expired, timeouts\rangle$

$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ cache,\ task_queue,$
$jobs,\ blob_storage,\ available_blobs,$
$worker_status,\ client_received,\ commit_count,$
$crashes,\ req_version,\ proxy,\ curr_req,$
$curr_epoch,\ curr_blob,\ curr_proxy\rangle$

$TimeoutInjector \triangleq InjectLoop$

$LeaseLoop \triangleq \land pc[\text{"LeaseManager"}] = \text{"LeaseLoop"}$
$\land \exists\, r \in ExpirableJobs :$
$\land$ IF $jobs[r].attempts < MaxAttempts$
THEN $\land jobs' = [jobs$ EXCEPT $![r] = [jobs[r]$ EXCEPT
$!.owner = NONE,$
$!.state = \text{"QUEUED"},$
$!.attempts = @ + 1,$
$!.lease_epoch = @ + 1,$
$!.blob_ptr = NONE,$
$!.rls_applied = \text{FALSE},$
$!.executed_predicate_tenant = NONE]]$
$\land task_queue' = Append(task_queue,\ r)$
ELSE $\land jobs' = [jobs$ EXCEPT $![r] = [jobs[r]$ EXCEPT
$!.owner = NONE,$
$!.state = \text{"FAILED"},$
$!.lease_epoch = @ + 1,$
$!.blob_ptr = NONE,$
$!.rls_applied = \text{FALSE},$
$!.executed_predicate_tenant = NONE]]$
$\land$ UNCHANGED $task_queue$
$\land lease_expired' = [lease_expired$ EXCEPT $![r] = \text{FALSE}]$
$\land pc' = [pc$ EXCEPT $![\text{"LeaseManager"}] = \text{"LeaseLoop"}]$
$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ cache,\ blob_storage,$
$available_blobs,\ worker_status,\ client_received,$
$commit_count,\ crashes,\ timeouts,\ req_version,$
$proxy,\ curr_req,\ curr_epoch,\ curr_blob,$
$curr_proxy\rangle$

$LeaseManager \triangleq LeaseLoop$

$CrashLoop \triangleq \land pc[\text{"Crasher"}] = \text{"CrashLoop"}$
$\land$ IF $crashes < MaxCrashes$
THEN $\land \exists\, w \in Workers : worker_status[w] = \text{"ACTIVE"}$
$\land \exists\, w \in \{x \in Workers : worker_status[x] = \text{"ACTIVE"}\} :$
$\land worker_status' = [worker_status$ EXCEPT $![w] = \text{"CRASHED"}]$
$\land crashes' = crashes + 1$
$\land pc' = [pc$ EXCEPT $![\text{"Crasher"}] = \text{"CrashLoop"}]$
ELSE $\land pc' = [pc$ EXCEPT $![\text{"Crasher"}] = \text{"Done"}]$
$\land$ UNCHANGED $\langle worker_status,\ crashes\rangle$
$\land$ UNCHANGED $\langle hash_ring,\ catalog_snapshot,\ cache,\ task_queue,$
$jobs,\ blob_storage,\ available_blobs,$
$client_received,\ commit_count,\ lease_expired,$
$timeouts,\ req_version,\ proxy,\ curr_req,$
$curr_epoch,\ curr_blob,\ curr_proxy\rangle$

$Crasher \triangleq CrashLoop$

$RecoverLoop \triangleq \land pc[\text{"Recoverer"}] = \text{"RecoverLoop"}$
$\land \exists\, w \in Workers : worker_status[w] = \text{"CRASHED"}$

$\land \exists\, w \in \{x \in \mathit{Workers} : \mathit{worker_status}[x] = \text{“CRASHED”}\} :$
$\mathit{worker_status}' = [\mathit{worker_status}$ EXCEPT $![w] = \text{“OFFLINE”}]$
$\land\ \mathit{pc}' = [\mathit{pc}$ EXCEPT $![\text{“Recoverer”}] = \text{“RecoverLoop”}]$
$\land$ UNCHANGED $\langle \mathit{hash_ring}, \mathit{catalog_snapshot}, \mathit{cache}, \mathit{task_queue},$
$\mathit{jobs}, \mathit{blob_storage}, \mathit{available_blobs},$
$\mathit{client_received}, \mathit{commit_count}, \mathit{lease_expired},$
$\mathit{crashes}, \mathit{timeouts}, \mathit{req_version}, \mathit{proxy}, \mathit{curr_req},$
$\mathit{curr_epoch}, \mathit{curr_blob}, \mathit{curr_proxy}\rangle$

$\mathit{Recoverer} \triangleq \mathit{RecoverLoop}$

$\mathit{GCLoop} \triangleq \land\ \mathit{pc}[\text{“GarbageCollector”}] = \text{“GCLoop”}$
$\land \exists\, b \in \mathit{GCable} :$
$\land\ \mathit{blob_storage}' = [\mathit{blob_storage}$ EXCEPT $![b] = \mathit{FreeBlob}]$
$\land\ \mathit{available_blobs}' = (\mathit{available_blobs} \cup \{b\})$
$\land\ \mathit{pc}' = [\mathit{pc}$ EXCEPT $![\text{“GarbageCollector”}] = \text{“GCLoop”}]$
$\land$ UNCHANGED $\langle \mathit{hash_ring}, \mathit{catalog_snapshot}, \mathit{cache}, \mathit{task_queue}, \mathit{jobs},$
$\mathit{worker_status}, \mathit{client_received}, \mathit{commit_count},$
$\mathit{lease_expired}, \mathit{crashes}, \mathit{timeouts}, \mathit{req_version}, \mathit{proxy},$
$\mathit{curr_req}, \mathit{curr_epoch}, \mathit{curr_blob}, \mathit{curr_proxy}\rangle$

$\mathit{GarbageCollector} \triangleq \mathit{GCLoop}$

$\mathit{EvictLoop} \triangleq \land\ \mathit{pc}[\text{“CacheEvictor”}] = \text{“EvictLoop”}$
$\land \exists\, e \in \mathit{CachedKeys} :$
$\mathit{cache}' = [\mathit{cache}$ EXCEPT $![e[1]][e[2]][e[3]][e[4]][e[5]] = \mathit{NoEntry}]$
$\land\ \mathit{pc}' = [\mathit{pc}$ EXCEPT $![\text{“CacheEvictor”}] = \text{“EvictLoop”}]$
$\land$ UNCHANGED $\langle \mathit{hash_ring}, \mathit{catalog_snapshot}, \mathit{task_queue}, \mathit{jobs},$
$\mathit{blob_storage}, \mathit{available_blobs}, \mathit{worker_status},$
$\mathit{client_received}, \mathit{commit_count}, \mathit{lease_expired},$
$\mathit{crashes}, \mathit{timeouts}, \mathit{req_version}, \mathit{proxy}, \mathit{curr_req},$
$\mathit{curr_epoch}, \mathit{curr_blob}, \mathit{curr_proxy}\rangle$

$\mathit{CacheEvictor} \triangleq \mathit{EvictLoop}$

$\mathit{ETLCommit} \triangleq \land\ \mathit{pc}[\text{“LakehouseETL”}] = \text{“ETLCommit”}$
$\land \exists\, t \in \{x \in \mathit{Tenants} : \mathit{catalog_snapshot}[x] < \mathit{MaxSnapshot}\} :$
$\land\ \mathit{cache}' = [p \in \mathit{ProxyNodes} \mapsto$
$[\mathit{cache}[p]$ EXCEPT $![t] =$
$[q \in \mathit{Queries} \mapsto$
$[v \in 1 \,..\, \mathit{MaxSnapshot} \mapsto$
$[s \in \mathit{SecCtxs} \mapsto$
IF $v < \mathit{catalog_snapshot}[t] + 1$
THEN $\mathit{NoEntry}$
ELSE $\mathit{cache}[p][t][q][v][s]]]]]]$
$\land\ \mathit{catalog_snapshot}' = [\mathit{catalog_snapshot}$ EXCEPT $![t] = \mathit{catalog_snapshot}[t] + 1]$
$\land\ \mathit{pc}' = [\mathit{pc}$ EXCEPT $![\text{“LakehouseETL”}] = \text{“ETLCommit”}]$
$\land$ UNCHANGED $\langle \mathit{hash_ring}, \mathit{task_queue}, \mathit{jobs}, \mathit{blob_storage},$
$\mathit{available_blobs}, \mathit{worker_status}, \mathit{client_received},$
$\mathit{commit_count}, \mathit{lease_expired}, \mathit{crashes}, \mathit{timeouts},$
$\mathit{req_version}, \mathit{proxy}, \mathit{curr_req}, \mathit{curr_epoch},$
$\mathit{curr_blob}, \mathit{curr_proxy}\rangle$

$\mathit{LakehouseETL} \triangleq \mathit{ETLCommit}$

$\mathit{Next} \triangleq \mathit{KedaAutoscaler} \lor \mathit{TimeoutInjector} \lor \mathit{LeaseManager} \lor \mathit{Crasher}$
$\lor \mathit{Recoverer} \lor \mathit{GarbageCollector} \lor \mathit{CacheEvictor} \lor \mathit{LakehouseETL}$

$\lor\ (\exists\, self \in Requests : ClientProcess(self))$
$\lor\ (\exists\, self \in Requests : ClientReleaser(self))$
$\lor\ (\exists\, self \in Workers : ComputeWorker(self))$

$Spec \triangleq \land Init \land \Box[Next]_{vars}$
$\land\ \forall\, self \in Requests : \mathrm{WF}_{vars}(ClientProcess(self))$
$\land\ \forall\, self \in Requests : \mathrm{WF}_{vars}(ClientReleaser(self))$
$\land\ \forall\, self \in Workers : \mathrm{WF}_{vars}(ComputeWorker(self))$
$\land\ \mathrm{WF}_{vars}(KedaAutoscaler)$
$\land\ \mathrm{WF}_{vars}(LeaseManager)$
$\land\ \mathrm{WF}_{vars}(Recoverer)$
$\land\ \mathrm{WF}_{vars}(GarbageCollector)$
$\land\ \mathrm{WF}_{vars}(CacheEvictor)$
$\land\ \mathrm{WF}_{vars}(LakehouseETL)$